

Amazon RDS PostgreSQL Production Setup – Multi-AZ Cluster with Writer and Reader Endpoints

To Begin with the Lab

Summary of the Lab

In this lab, a production-style **Amazon RDS** PostgreSQL cluster was created using the **Production** template instead of the earlier free-tier setup. The cluster deployed one writer instance and two reader instances across different Availability Zones so the database could support both high availability and read scaling. A jump server was used to connect to the database with the PostgreSQL client, and sample database objects were created on the writer endpoint. The reader endpoint was then tested to confirm that it allowed read operations but rejected write attempts. Finally, the lab verified that the cluster exposed separate writer and reader endpoints and could fail over automatically if the primary zone became unavailable.

Understanding Amazon RDS Multi-AZ Cluster with Writer and Reader Endpoints

A production **Amazon RDS** Multi-AZ cluster exists so a database can stay available even when one Availability Zone has a problem. The writer endpoint sends all insert, update, and delete activity to the primary writer instance, while the reader endpoint distributes read queries across the reader nodes. This setup improves performance because reads and writes are separated instead of all traffic hitting one instance. It also improves availability because AWS can promote a reader node if the writer-side Availability Zone fails.

Example:

An application writes new orders through the writer endpoint and runs reporting queries through the reader endpoint. If the writer's Availability Zone fails, AWS automatically promotes a reader node and keeps the application online. That means the app continues working without manual intervention or downtime.

Lab Steps

Step 1 – Create the Production Cluster

Create a PostgreSQL RDS cluster using the **Production** template and the Multi-AZ cluster option.

- Sign in to the AWS Management Console.
- Open **Amazon RDS**.
- Below Create with full configuration and Click **Create**.

Welcome to Aurora and RDS [Info](#)

Amazon Relational Database Service (Amazon RDS) and Amazon Aurora are fully managed database services that simplify setup, operation, and scaling of relational databases in the cloud.



Create with express configuration in seconds

Create and query an Aurora PostgreSQL serverless database with pre-configured settings to get started quickly.

[Create](#)



Create with full configuration

Create a database with custom configurations and enable features you want across a variety of engines including Aurora (compatible with MySQL & PostgreSQL), MySQL, PostgreSQL, MariaDB, Oracle, Microsoft SQL Server, and IBM Db2. Or use an existing database using restore from S3.

[Create](#) [Restore from S3](#)

Service overview

Viewing data from Asia Pacific (Mumbai) (ap-south-1) Region.

DB clusters 0 **DB instances** 0 **Snapshots** 5 **Recent events** 0

[View service quota](#)

Explore Aurora & RDS

In this activity, you will learn how to create a database. To begin, choose [Start tutorial](#).

Estimated duration 2-5 minutes

[Start tutorial](#)

Service health

Current status [View service health dashboard](#)

RDS costs

- Select any engine but for this lab I have chosen **PostgreSQL**.

Availability zone group [Info](#)

Availability zone group

Select the availability zone where you want to create your database.

US East (N. Virginia)

Engine options

Engine type [Info](#)

☐ Aurora (MySQL Compatible)
 ☐ Aurora (PostgreSQL Compatible)
 ☐ MySQL
 ☒ PostgreSQL

☐ MariaDB
 ☐ Oracle
 ☐ Microsoft SQL Server
 ☐ IBM Db2

Choose a database creation method

☒ Full configuration
 ☐ Easy create

For select all the configuration options, including ones for availability, security, backups, and maintenance.

Use recommended best-practice configurations. Some configuration options can be changed after the database is created.

- Choose **Full Configuration**.
- Select the **Production** template.
- Choose the Multi-AZ cluster deployment option with one writer and two reader instances (3 instances).

aws [Search] [Alt+S] United States (N. Virginia) cloud:reshtk (463646775279) cf-shashank1

Aurora and RDS > Databases > Create database

Choose a database creation method

☒ **Full configuration**
You set all of the configuration options, including ones for availability, security, backups, and maintenance.

☐ **Easy create**
Use recommended best-practice configurations. Some configuration options can be changed after the database is created.

Templates

Choose a sample template to meet your use case.

☒ **Production**
Use templates for high availability and fast, consistent performance.

☐ **Dev/Test**
This instance is intended for development use outside of a production environment.

☐ **Free tier**
Use RDS Free Tier to develop new applications, test existing applications, or gain hands-on experience with Amazon RDS. [Info](#)

Availability and durability

Deployment options [Info](#)
Choose the deployment option that provides the availability and durability needed for your use case. AWS is committed to a certain level of uptime depending on the deployment option you choose. Learn more in the [Amazon RDS service level agreement \(SLA\)](#).

☒ **Multi-AZ DB cluster deployment (3 instances)**
Creates a primary DB instance with two readable standby instances in separate Availability Zones. This setup provides:
• 99.95% uptime
• Redundancy across Availability Zones
• Increased read capacity
• Reduced write latency

☐ **Multi-AZ DB instance deployment (2 instances)**
Creates a primary DB instance with a non-readable standby instance in a separate Availability Zone. This setup provides:
• 99.95% uptime
• Redundancy across Availability Zones

☐ **Single-AZ DB instance deployment (1 instance)**
Creates a single DB instance without standby instances. This setup provides:
• 99.5% uptime
• No data redundancy

Write/read endpoint AZ 1 **Reader endpoints** AZ 2 **Write/read endpoint** AZ 1 **Standby (no endpoint)** AZ 2

- Choose correct PostgreSQL Version.

aws [Search] [Alt+S] United States (N. Virginia) cloud:reshtk (463646775279) cf-shashank1

Aurora and RDS > Databases > Create database

Engine version

[Info](#)
View the engine versions that support the following database features.

Hide filters

☒ **Show only versions that support the Multi-AZ DB cluster** [Info](#)
Create a Multi-AZ DB cluster with one primary DB instance and two readable standby DB instances. Multi-AZ DB clusters provide up to 2x faster transaction commit latency and automatic failover in typically under 35 seconds.

Engine version
PostgreSQL 18.2-R1

☐ **Enable RDS Extended Support** [Info](#)
Amazon RDS Extended Support is a paid offering. By selecting this option, you consent to being charged for this offering if you are running your database major version past the RDS end of standard support date for that version. Check the end of standard support date for your major version in the [RDS for PostgreSQL documentation](#).

DB instance identifier [Info](#)
Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.
database-2
The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 63 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

Credentials Settings

Master username [Info](#)
Type a login ID for the master user of your DB instance.
postgres

Credentials management
You can use AWS Secrets Manager or manage your master user credentials.

☐ **Managed in AWS Secrets Manager - most secure**
RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.

☒ **Self managed**
You must create a password or have RDS create a password that you manage.

- Enter the database identifier and credentials.

Aurora and RDS > Databases > Create database

Master username [Info](#)
Type a login ID for the master user of your DB cluster.
postgres

1 to 16 alphanumeric characters. The first character must be a letter.

Credentials management
You can use AWS Secrets Manager or manage your master user credentials.

☐ Managed in AWS Secrets Manager - *most secure*
RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.

☒ Self managed
Create your own password or have RDS create a password that you manage.

☐ Auto generate password
Amazon RDS can generate a password for you, or you can specify your own password.

Master password [Info](#)

Password strength Weak
Minimum constraints: At least 8 printable ASCII characters. Can't contain any of the following symbols: / * @

Confirm master password [Info](#)

▼ **Additional credentials settings**

Database authentication options [Info](#)

☒ Password authentication
Authenticates using database passwords.

☐ Password and IAM database authentication
Authenticates using the database password and user credentials through AWS IAM users and roles.

☐ Password and Kerberos authentication (not available for Multi-AZ DB cluster)
Choose a directory in which you want to allow authorized users to authenticate with this DB instance using Kerberos Authentication.

- Select a suitable instance class and storage size but for lab purpose we are selecting lower ones as shown.

aws Search [Alt+S] Ask Amazon Q United States (N. Virginia) cf-shashank1

Aurora and RDS > Databases > Create database

Storage

Storage type [Info](#)
Provisioned IOPS SSD (io2) storage volumes are now available.
General Purpose SSD (gp3)
Performance scales independently from storage.

Allocated storage [Info](#)
20 GiB
Minimum 20 GiB. Maximum 16,384 GiB.

Provisioned IOPS [Info](#)
3000 IOPS
Baseline IOPS of 3,000 IOPS is included for allocated storage less than 400 GiB.

Storage throughput [Info](#)
125 MiBps
Baseline storage throughput of 125 MiBps is included for allocated storage less than 400 GiB.

To provision additional IOPS and throughput, increase the allocated storage to 400 GiB or greater.

► **Additional storage configuration**

Connectivity [Info](#)

Compute resource

- Let AWS auto-generate the password or set one manually.
- Choose the appropriate VPC and database subnet group.
- Keep **Public access** set to **No**.
- Attach the RDS security group created for the lab.

Virtual private cloud (VPC) Info
Choose the VPC. The VPC defines the virtual networking environment for this DB cluster.
Default VPC (vpc-099792461c7d0a616)
11 subnets, 7 availability zones
After a database is created, you can't change its VPC. Only VPCs with a corresponding DB subnet group are listed.

DB subnet group Info
Choose the DB subnet group. The DB subnet group defines which subnets and IP ranges the DB cluster can use in the VPC that you selected.
default-vpc-099792461c7d0a616

Public access Info
☐ Yes
RDS assigns a public IP address to the cluster. Amazon EC2 instances and other resources outside of the VPC can connect to your cluster. Resources inside the VPC can also connect to the cluster. Choose one or more VPC security groups that specify which resources can connect to the cluster.
☒ No
RDS doesn't assign a public IP address to the cluster. Only Amazon EC2 instances and other resources inside the VPC can connect to your cluster. Choose one or more VPC security groups that specify which resources can connect to the cluster.

VPC security group (firewall) Info
Choose one or more VPC security groups to allow access to your database. Make sure that the security group rules allow the appropriate incoming traffic.
☒ Choose existing
Choose existing VPC security groups
☐ Create new
Create new VPC security group

Existing VPC security groups
Choose one or more options
RDS-SG X default X

- Review the estimated monthly cost.
- Click **Create database**.

Step 2 – Wait for the Cluster and Verify the Topology

Wait until the cluster becomes available and confirm that the writer and reader nodes are placed in different Availability Zones.

- Wait for the cluster status to change from **Creating** or **Modifying** to **Available**.
- Open the cluster details page.
- Verify that the cluster contains one writer instance and two reader instances.
- Confirm that the configuration shows **Multi-AZ = Yes**.
- Check that the writer and reader nodes are spread across different Availability Zones.

DB identifier	Status	Role	Engine	Region & AZ	Size	Recommendations	CPU
database-2	Available	Multi-AZ DB cluster	PostgreSQL	us-east-1	3 instances	1 Informational	-
database-2-instance-1	Available	Writer instance	PostgreSQL	us-east-1c	db.c6gd...		
database-2-instance-2	Available	Reader instance	PostgreSQL	us-east-1a	db.c6gd...		
database-2-instance-3	Available	Reader instance	PostgreSQL	us-east-1f	db.c6gd...		

- Note the writer endpoint and reader endpoint shown in the console.

The screenshot shows the AWS Aurora and RDS console for a database instance named 'database-2'. The 'Endpoints' tab is selected, displaying a table of endpoints. The table has columns for 'Endpoint name', 'Status', 'Type', and 'Port'. Two endpoints are listed: a 'Writer' endpoint and a 'Reader' endpoint, both with a status of 'Available' and a port of 5432. The 'Writer' endpoint name is highlighted with a red box.

Endpoint name	Status	Type	Port
database-2.cluster-cghaxz7pqg8a.us-east-1.rds.amazonaws.com	Available	Writer	5432
database-2.cluster-ro-cghaxz7pqg8a.us-east-1.rds.amazonaws.com	Available	Reader	5432

Step 3 – Connect to the Writer Endpoint from the Jump Server

Use the jump server and PostgreSQL client to connect to the writer endpoint, then create a sample database and table.

This screenshot is a zoomed-in view of the 'Endpoints' tab in the AWS Aurora and RDS console. It shows the same table as the previous screenshot, with the 'Writer' endpoint name highlighted by a red box.

Endpoint name	Status	Type	Port
database-2.cluster-cghaxz7pqg8a.us-east-1.rds.amazonaws.com	Available	Writer	5432
database-2.cluster-ro-cghaxz7pqg8a.us-east-1.rds.amazonaws.com	Available	Reader	5432

- Open the jump server.
- Connect to the jump server using SSH.


```
root@ip-172-31-20-58:~  
[root@ip-172-31-20-58 ~]# sudo dnf update  
Last metadata expiration check: 0:01:56 ago on Wed Jun 17 09:41:59 2026.  
Dependencies resolved.  
Nothing to do.  
Complete!  
[root@ip-172-31-20-58 ~]# sudo dnf install -y postgresql18  
Last metadata expiration check: 0:02:10 ago on Wed Jun 17 09:41:59 2026.  
Dependencies resolved.  
=====
```

Package	Architecture	Version	Repository	Size
Installing: postgresql18	x86_64	18.4-1.amzn2023.0.1	amazonlinux	2.0 M
Installing dependencies: postgresql18-private-libs	x86_64	18.4-1.amzn2023.0.1	amazonlinux	160 k

```
=====
```

Transaction Summary

=====

Install 2 Packages

Total size: 2.1 M
Installed size: 8.9 M
Downloading Packages:
[SKIPPED] postgresql18-18.4-1.amzn2023.0.1.x86_64.rpm: Already downloaded
[SKIPPED] postgresql18-private-libs-18.4-1.amzn2023.0.1.x86_64.rpm: Already downloaded
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
Preparing :
Installing : postgresql18-private-libs-18.4-1.amzn2023.0.1.x86_64 1/1
Installing : postgresql18-18.4-1.amzn2023.0.1.x86_64 1/2
Installing : postgresql18-18.4-1.amzn2023.0.1.x86_64 2/2
Running scriptlet: postgresql18-18.4-1.amzn2023.0.1.x86_64 2/2
Verifying : postgresql18-18.4-1.amzn2023.0.1.x86_64 1/2
Verifying : postgresql18-private-libs-18.4-1.amzn2023.0.1.x86_64 2/2

Installed:
postgresql18-18.4-1.amzn2023.0.1.x86_64
postgresql18-private-libs-18.4-1.amzn2023.0.1.x86_64

Complete!

- Connect to the writer endpoint using the master credentials.
- `psql -h {endpoint} -U {username}`

```
root@ip-172-31-20-58:~  
[root@ip-172-31-20-58 ~]# psql -h database-2.cluster-cghaxz7pqg8a.us-east-1.rds.amazonaws.com -U postgres  
Password for user postgres:  
psql (18.4, server 18.2)  
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)  
Type "help" for help.  
postgres=>
```

- Create a sample database.
- Switch to the new database.
- Create a sample table for testing.

CREATE DATABASE demo;

\c demo

CREATE TABLE employees (
id SERIAL PRIMARY KEY,


```
name VARCHAR(100) NOT NULL,  
position VARCHAR(100)  
);  
  
INSERT INTO employees (name, position)  
VALUES ('John Doe', 'Software Engineer');  
  
INSERT INTO employees (name, position)  
VALUES ('Jane Smith', 'Project Manager');  
  
SELECT * FROM employees;
```

```
postgres=> CREATE DATABASE demo;  
CREATE DATABASE  
postgres=> \c demo  
psql (18.4, server 18.2)  
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)  
You are now connected to database "demo" as user "postgres".  
demo=> CREATE TABLE employees (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    position VARCHAR(100)  
);  
CREATE TABLE  
demo=> INSERT INTO employees (name, position)  
VALUES ('John Doe', 'Software Engineer');  
INSERT 0 1  
demo=> INSERT INTO employees (name, position)  
VALUES ('Jane Smith', 'Project Manager');  
INSERT 0 1  
demo=> SELECT * FROM employees;  
 id | name      | position  
----+-----+-----  
  1 | John Doe  | Software Engineer  
  2 | Jane Smith | Project Manager  
(2 rows)  
  
demo=> |
```

- Verify that the write operations succeed on the writer endpoint.

Step 4 – Test the Reader Endpoint

Connect to the reader endpoint and confirm that it is read-only.

Connectivity & security | Monitoring | Logs & events | Configuration | Maintenance & backups | Tags | Recommendations

Connect using [Info](#)

☐ Code snippets
Use when connecting through SDK, APIs, or third-party tools including agents.

☐ CloudShell
Use for a quick access to AWS CLI that launches directly from the AWS Management Console.

☒ Endpoints
Use when connecting through any IDE interface.

Database name: postgres Master username: postgres Internet access gateway: Disabled Port: 5432

Endpoint type: Cluster endpoint

Endpoints (2)

Filter resources

Endpoint name	Status	Type	Port
database-2.cluster-cghaxz7pqg8a.us-east-1.rds.amazonaws.com	Available	Writer	5432
database-2.cluster-ro-cghaxz7pqg8a.us-east-1.rds.amazonaws.com	Available	Reader	5432

- Open a new PostgreSQL client session.
- Connect to the reader endpoint instead of the writer endpoint.

```
[root@ip-172-31-20-58 ~]# psql -h database-2.cluster-ro-cghaxz7pqg8a.us-east-1.rds.amazonaws.com -U postgres
Password for user postgres:
psql (18.4, server 18.2)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type "help" for help.

postgres=> |
```

- Change database to demo.
- Run a SELECT query on the sample table.

```
SELECT * FROM employees;
```

- Verify that the data is returned successfully.
- Attempt an INSERT statement on the reader endpoint.

```
INSERT INTO employees (name, position)
VALUES ('Test User', 'Analyst');
```

- Confirm that the write attempt fails with a read-only transaction error.

```
[root@ip-172-31-20-58 ~]# psql -h database-2.cluster-ro-cghaxz7pqg8a.us-east-1.rds.amazonaws.com -U postgres
Password for user postgres:
psql (18.4, server 18.2)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type "help" for help.

postgres=> \c demo
psql (18.4, server 18.2)
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
You are now connected to database "demo" as user "postgres".
demo=> SELECT * FROM employees;
 id | name | position
----+-----+-----
  1 | John Doe | Software Engineer
  2 | Jane Smith | Project Manager
(2 rows)

demo=> INSERT INTO employees (name, position)
VALUES ('Test User', 'Analyst');
ERROR:  cannot execute INSERT in a read-only transaction
demo=> |
```

- Verify that the reader endpoint is intended only for read traffic.

Step 5 – Review Failover and Clean Up

Review how AWS handles failover automatically and then remove the cluster after the demo.

- Before failover database-2-instance-1 was writer instance.

The screenshot shows the AWS Aurora and RDS console for a database cluster named 'database-2'. The 'Related' table lists the following instances:

DB Identifier	Status	Role	Engine	Region	Size	Recommendations
database-2	Available	Multi-AZ ...	PostgreSQL	us-east-1	3 instances	1 Informational
database-2-instance-1	Available	Writer ins...	PostgreSQL	us-east-1c	db.c6gd...	
database-2-instance-2	Available	Reader ins...	PostgreSQL	us-east-1a	db.c6gd...	
database-2-instance-3	Available	Reader ins...	PostgreSQL	us-east-1f	db.c6gd...	

The 'Actions' menu is open, showing options like Reboot, Delete, Failover, Set up EC2 connection, Set up Lambda connection, Migrate data to this database, Create read replica, Promote, Take snapshot, Restore to point in time, and Create ElastiCache cluster.

- After failover AWS picked instance-2 to become the new writer automatically (promoted) and the instance-1 demoted to reader.

The screenshot shows the AWS Aurora and RDS console for a database instance named 'database-2-instance-1'. A green banner at the top indicates 'Successfully failed over DB cluster database-2.' The 'Related' table lists the following instances:

DB Identifier	Status	Role	Engine	Region	Size	Recommendations	CPU
database-2	Available	Multi-AZ DB ...	PostgreSQL	us-east-1	3 instances	1 Informational	-
database-2-instance-1	Available	Reader insta...	PostgreSQL	us-east-1c	db.c6gd...		25%
database-2-instance-2	Available	Writer instance	PostgreSQL	us-east-1a	db.c6gd...		30%
database-2-instance-3	Available	Reader insta...	PostgreSQL	us-east-1f	db.c6gd...		25%

The 'Actions' menu is open, showing options like Reboot, Delete, Failover, Set up EC2 connection, Set up Lambda connection, Migrate data to this database, Create read replica, Promote, Take snapshot, Restore to point in time, and Create ElastiCache cluster.

- Select database and click on **Modify**.

database-2

Related

DB identifier	Status	Role	Engine	Region	Size	Recommendations	CPU
database-2	Available	Multi-AZ DB cl...	PostgreSQL	us-east-1	3 instances	1 Informational	-
database-2-instance-1	Available	Reader instance	PostgreSQL	us-east-1c	db.c6gd...		29.92
database-2-instance-2	Available	Writer instance	PostgreSQL	us-east-1a	db.c6gd...		31.30
database-2-instance-3	Available	Reader instance	PostgreSQL	us-east-1f	db.c6gd...		26.50

Connectivity & security | Monitoring | Logs & events | Configuration | Maintenance & backups | Tags | Recommendations

Connect using [Info](#)

- ☒ Code snippets: Use when connecting through SDK, APIs, or third-party tools including agents.
- ☐ CloudShell: Use for a quick access to AWS CLI that launches directly from the AWS Management Console.
- ☐ Endpoints: Use when connecting through any IDE interface.

Internet access gateway: Disabled

IAM Authentication: Disabled

Programming language | Endpoint type | Connect to

- Scroll down and disable deletion protection if it is enabled.

Modify DB cluster: database-2

The number of days (1-35) for which automatic backups are kept: 7 days

Backup window [Info](#)

Select the period for which you want automated backups of the DB cluster to be created by Amazon RDS.

☐ Choose a window

☒ No preference

Start time: 06:01 UTC

Duration: 0.5 hours

☒ Copy tags to snapshots

Maintenance

Auto minor version upgrade [Info](#)

☒ Enable auto minor version upgrade

Enabling auto minor version upgrade will automatically upgrade your database minor version. For limitations and more details, see [Automatically upgrading the minor engine version documentation](#).

DB cluster maintenance window

The weekly time range during which system maintenance can occur.

Start day: Wednesday

Start time: 08:31 UTC

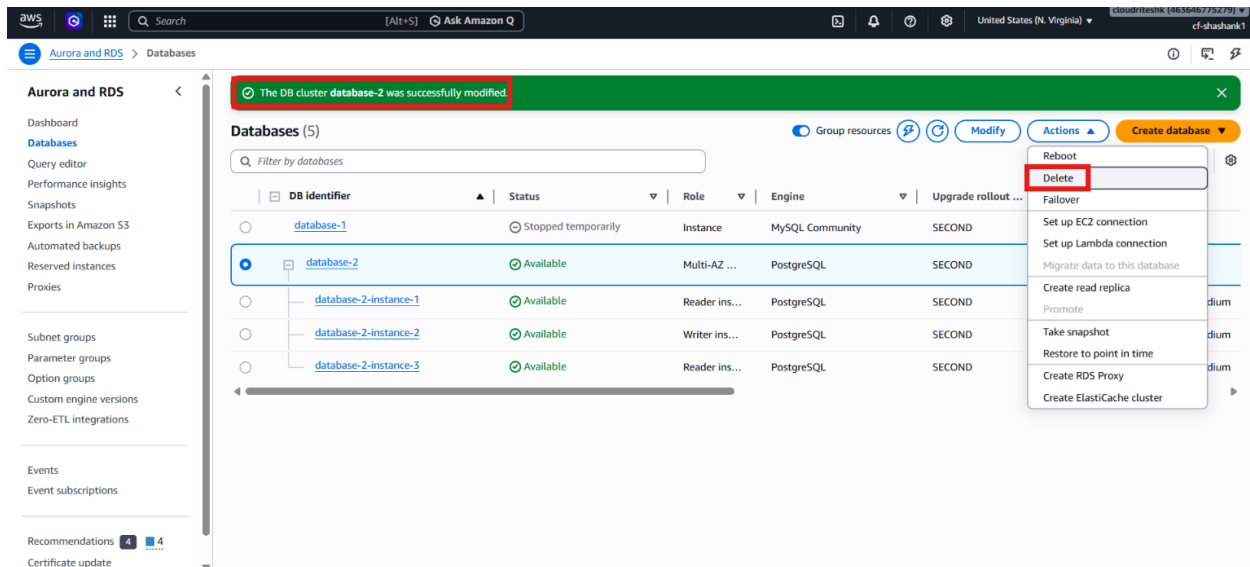
Duration: 0.5 hours

☐ Enable deletion protection

Protects the database from being deleted accidentally. While this option is enabled, you can't delete the database cluster.

Cancel **Continue**

- Wait for the change to apply.
- Delete the cluster after the lab is complete.



- Confirm that the cluster is removed successfully.

Verification

- The PostgreSQL RDS cluster was created successfully.
- The cluster used the **Production** template.
- The cluster included one writer node and two reader nodes.
- The writer endpoint accepted write operations.
- The reader endpoint allowed reads and rejected writes.
- The cluster was configured for Multi-AZ high availability.